UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL

CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO– JULIO 2026

I. PORTADA

Tema: Evaluacion UNIDAD 1

Unidad de Organización Curricular: BÁSICA

Nivel y Paralelo: 1ro Software B

Alumnos participantes: Rocha Rocha Carolina Estefania

Segovia Garcia Joseph Andre

Barrionuevo Montesdeoca Job Gabriel

Asignatura: lgoritmos y Lógica de programación

Docente: Ing. Jose Ruben Caizabuano, Mg.

II. INFORME DE EVALUACION

2.1 Objetivos

General:
Desarrollar un sistema automatizado en C++ y PSeInt para evaluar la calidad de proyectos de software, integrando métricas técnicas, cumplimiento de formalidades y restricciones de avance mediante estructuras de control condicional.
Específicos:

- Validar el rango de notas de entrada (0 a 10) para asegurar la consistencia del promedio.
- Aplicar ajustes por errores y formalidad para determinar una calificación final ponderada.
- Clasificar el estado del proyecto según la nota obtenida y las restricciones de avance real.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 1

No presenciales: 0

2.4 Instrucciones

Indicaciones para el estudiante:

- Desarrolle los 2 ejercicios propuestos aplicando la metodología de programación.
- En cada ejercicio incluya: análisis del problema, entradas, procesos y salidas, pseudocódigo en PSeInt, prueba de escritorio, implementación en C++ y evidencia de ejecución.
- Utilice nombres de variables claros, valide rangos cuando corresponda y muestre resultados comprensibles en pantalla.
- La nota total es sobre 2,00 puntos: cada ejercicio vale 1,00 punto.

2.5 Listado de equipos, materiales y recursos

Inteligencia artificial, TAC
- Computador o laptop.
- Plataforma educativa de la UTA.
- Internet para consultas.
- Papel
- Material de escritorio

.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas
☒Simuladores y laboratorios virtuales
☒Aplicaciones educativas
☐Recursos audiovisuales
☐Gamificación
☒Inteligencia Artificial

Otros (Especifique): Lenguaje de programación java,IDE para programar(Neatbeans)

2.6 Actividades por desarrollar

- Analice la información de el ejercicio propuesto
- Realice el diseño, codificación y posteriormente compilar, ejecutar y verificar el resultado.
- Subir a la plataforma el/los archivo(s) solicitados hasta la fecha indicada

2.7. Desarrollo / Anexos

1) **ENUNCIADO**

Ejercicio 2. Evaluación de calidad de un proyecto de software

Se desea evaluar el resultado de un proyecto desarrollado por un estudiante, considerando rendimiento técnico,
formalidad y restricciones del caso.

Datos de entrada

• Nota de análisis
• Nota de diseño
• Nota de codificación
• Porcentaje de avance real
• Número de errores detectados
• Presentó documentación completa (Sí/No)
• Realizó exposición final (Sí/No)

Proceso requerido

1. Calcular el promedio técnico de las tres notas.
2. Validar que cada nota esté entre 0 y 10.
3. Restar 0,5 puntos al promedio por cada error detectado.
4. Si presentó documentación completa, sumar 0,5 puntos.
5. Si realizó exposición final, sumar 0,5 puntos.
6. Obtener la nota final.
7. Si la nota final es mayor que 10, dejarla en 10.
8. Si la nota final es menor que 0, dejarla en 0.
9. Asignar estado: 9 a 10 Excelente; 7 a 8,99 Aprobado; 5 a 6,99 Recuperación; menor a 5 Reprobado.
10. Si el avance real es menor al 60 %, no puede quedar en Excelente.
11. Si tiene buena nota pero no presentó documentación, mostrar la observación: "Buen producto, pero mala formalidad".

Salida esperada

• Promedio técnico
• Ajustes aplicados
• Nota final
• Estado
• Observación

2) ANÁLISIS DEL PROBLEMA

El problema consiste en desarrollar un programa que permita evaluar la calidad de un proyecto de software realizado por un estudiante, tomando en cuenta su rendimiento técnico, el cumplimiento de requisitos formales y ciertas restricciones establecidas en el caso.

El programa debe solicitar como datos de entrada las notas correspondientes a las tres etapas principales del desarrollo del proyecto: análisis, diseño y codificación. Estas notas representan el desempeño técnico del estudiante, por lo que es necesario validar que cada una se encuentre dentro del rango permitido de 0 a 10. En caso de que alguna nota esté fuera de este rango, el sistema deberá considerar el dato como inválido y no continuar con el proceso hasta corregirlo, garantizando así la consistencia de la información.

Además, se debe ingresar el porcentaje de avance real del proyecto, el número de errores detectados, y dos variables lógicas que indiquen si el estudiante presentó la documentación completa y si realizó la exposición final del proyecto. Estos elementos permiten evaluar no solo la calidad técnica, sino también la formalidad y cumplimiento del trabajo.

Durante el procesamiento, el programa calculará el promedio técnico utilizando las tres notas ingresadas. Este promedio servirá como base para determinar la calificación del proyecto. Posteriormente, se aplicarán una serie de **ajustes que actúan como penalizaciones y bonificaciones**:

- Por cada error detectado, se restarán 0,5 puntos al promedio, lo cual representa una penalización por fallas en el desarrollo.

- Si el estudiante presentó la documentación completa, se sumarán 0,5 puntos como reconocimiento al cumplimiento formal.
- Si realizó la exposición final, también se sumarán 0,5 puntos como incentivo a la correcta presentación del proyecto.

Estos ajustes permiten obtener una nota final más justa, considerando tanto aspectos técnicos como formales.

El sistema debe aplicar **restricciones de límites**, asegurando que la nota final no supere el valor máximo de 10 ni sea menor que 0. En caso de que los cálculos excedan estos límites, se deben ajustar automáticamente al rango permitido.

Una vez obtenida la nota final, el programa procederá a clasificar el resultado del estudiante en uno de los siguientes estados:

- De 9 a 10: Excelente

- De 7 a 8,99: Aprobado
- De 5 a 6,99: Recuperación
- Menor a 5: Reprobado

Sin embargo, se debe aplicar una **regla especial importante**: si el porcentaje de avance real del proyecto es menor al 60%, el estudiante no podrá ser clasificado como "Excelente", incluso si su nota final lo permite. En este caso, su estado deberá ajustarse a "Aprobado", reflejando que no cumplió completamente con el desarrollo esperado.

Adicionalmente, el programa debe generar una observación en caso de que el estudiante haya obtenido una buena calificación (nota final mayor o igual a 7), pero no haya presentado la documentación completa. En esta situación, se mostrará el mensaje:
"Buen producto, pero mala formalidad", indicando que, aunque el resultado técnico fue adecuado, no cumplió con los requisitos formales del proyecto.

Finalmente, el programa mostrará como salida:

- El promedio técnico calculado inicialmente.
- El valor total de los ajustes aplicados (bonificaciones y penalizaciones).
- La nota final obtenida.
- El estado asignado.
- La observación correspondiente, si aplica.

De esta manera, el programa permite realizar una evaluación integral del proyecto, considerando no solo el rendimiento técnico, sino también aspectos de calidad, formalidad y cumplimiento de condiciones establecidas.

3) DECLARACIÓN DE VARIABLES

Variables de Entrada (Datos solicitados)

- **analisis:** Almacena la nota de la etapa de análisis (Real/Punto flotante).
- **diseno:** Almacena la nota de la etapa de diseño (Real/Punto flotante).
- **codificacion:** Almacena la nota de la etapa de codificación (Real/Punto flotante).
- **avance:** Porcentaje de desarrollo real del proyecto (Real/Punto flotante).
- **errores:** Cantidad de fallas detectadas para aplicar penalización (Entero).
- **presento_doc:** Variable lógica/indicador para la documentación completa (Lógico o Entero 1/0).
- **hizo_expo**: Variable lógica/indicador para la exposición final (Lógico o Entero 1/0).

Variables de Proceso (Cálculos internos)

- **promedio_tecnico:** Resultado del cálculo de las tres notas iniciales (Real).
- **ajustes_totales:** Acumulador de la suma de bonificaciones y resta de penalizaciones (Real).
- **penalizacion_errores:** Cálculo de los puntos a restar según el número de errores (Real).

Variables de Salida (Resultados finales)

- nota_final: Calificación definitiva ajustada a los límites de 0 a 10 (Real).
- estado: Clasificación cualitativa (Excelente, Aprobado, Recuperación o Reprobado) (Cadena de texto).
- **observacion:** Mensaje preventivo sobre la formalidad del proyecto (Cadena de texto).

4) ALGORITMO PSEINT

1. **Inicio**.
2. Declarar las variables para notas (análisis, diseño, codificación), avance, errores, documentación, exposición, promedio, ajustes, nota final, estado y observación.
3. Solicitar al usuario el ingreso de las tres notas técnicas (análisis, diseño y codificación).
4. Solicitar el porcentaje de avance, el número de errores detectados y la confirmación (1 o 0) de documentación y exposición.
5. **Validar las notas**:
   5.1. Si alguna nota es menor a 0 o mayor a 10, mostrar mensaje de error y **Finalizar**.
6. Calcular el **Promedio Técnico** sumando las tres notas y dividiendo para tres.
7. Inicializar la nota_final con el valor del promedio y los ajustes en cero.
8. **Aplicar penalización por errores**: Multiplicar el número de errores por 0.5 y restar ese valor tanto a la nota_final como a los ajustes.
9. **Aplicar bonificación por formalidad**:
   - Si presentó documentación (1), sumar 0.5 a la nota_final y a los ajustes.
   - Si realizó exposición (1), sumar 0.5 a la nota_final y a los ajustes.
10. **Ajustar límites de la nota**:
    - Si nota_final es mayor a 10, fijarla en 10.
    - Si nota_final es menor a 0, fijarla en 0.
11. **Asignar Estado inicial**:
    - Si es >= 9: "Excelente".
    - Si está entre 7 y 8.99: "Aprobado".
    - Si está entre 5 y 6.99: "Recuperación".
    - Si es < 5: "Reprobado".
12. **Validar restricción de avance**: Si el estado es "Excelente" pero el avance es menor al 60%, cambiar el estado a "Aprobado".
13. **Generar Observación**: Si la nota final es satisfactoria (>= 7) pero no entregó documentación, asignar el mensaje: "Buen producto, pero mala formalidad".

14. Mostrar el Promedio Técnico, los Ajustes totales, la Nota Final, el Estado y la Observación.
15. **Fin**.

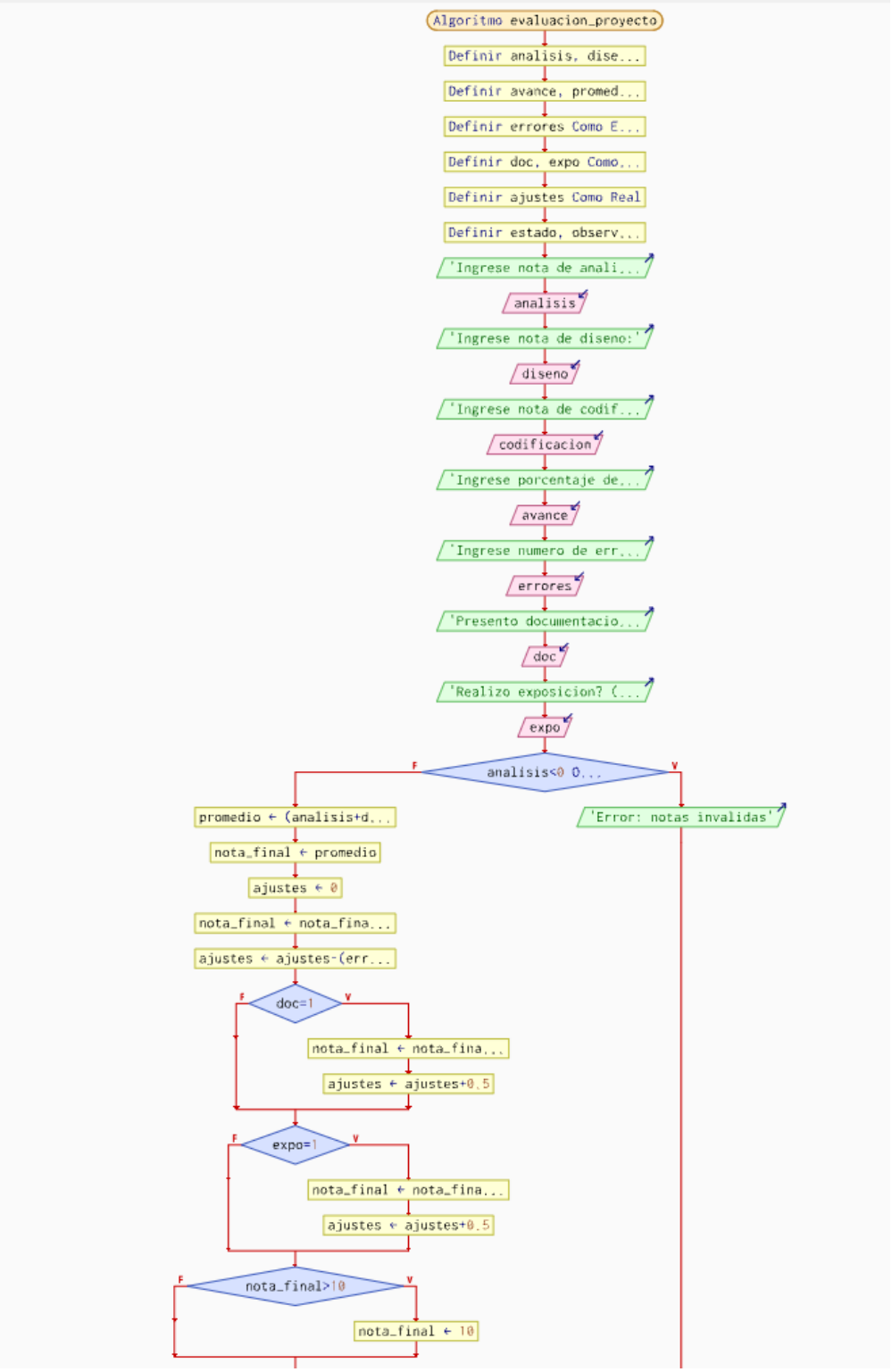
5) PSEUDOCODIGO EN PSEINT

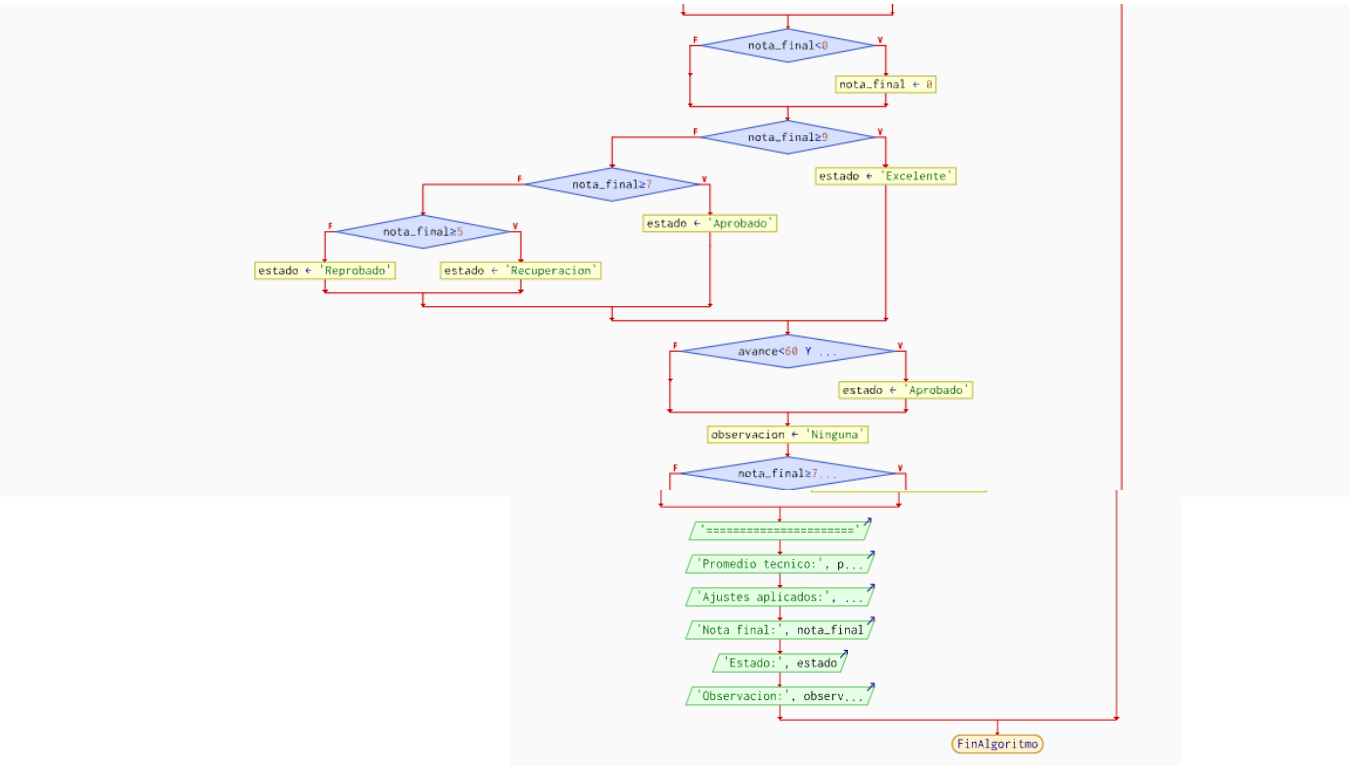
```
Algoritmo evaluacion_proyecto

Definir analisis, diseno, codificacion Como Real
Definir avance, promedio, nota_final Como Real
Definir errores Como Entero
Definir doc, expo Como Entero
Definir ajustes Como Real
Definir estado, observacion Como Cadena


Escribir "Ingrese nota de analisis:"
Leer analisis

Escribir "Ingrese nota de diseno:"
Leer diseno

Escribir "Ingrese nota de codificacion:"
Leer codificacion

Escribir "Ingrese porcentaje de avance:"
Leer avance

Escribir "Ingrese numero de errores:"
Leer errores

Escribir "Presento documentacion? (1=Si, 0=No):"
Leer doc

Escribir "Realizo exposicion? (1=Si, 0=No):"
Leer expo


Si analisis < 0 O analisis > 10 O diseno < 0 O diseno > 10 O codificacion < 0 O codificacion > 10 Entonces
  Escribir "Error: notas invalidas"
Sino


  promedio <- (analisis + diseno + codificacion) / 3
```

```
nota_final <- promedio
ajustes <- 0


nota_final <- nota_final - (errores * 0.5)
ajustes <- ajustes - (errores * 0.5)


Si doc = 1 Entonces
  nota_final <- nota_final + 0.5
  ajustes <- ajustes + 0.5
FinSi


Si expo = 1 Entonces
  nota_final <- nota_final + 0.5
  ajustes <- ajustes + 0.5
FinSi


Si nota_final > 10 Entonces
  nota_final <- 10
FinSi

Si nota_final < 0 Entonces
  nota_final <- 0
FinSi


Si nota_final >= 9 Entonces
  estado <- "Excelente"
Sino
  Si nota_final >= 7 Entonces
    estado <- "Aprobado"
  Sino
    Si nota_final >= 5 Entonces
      estado <- "Recuperacion"
    Sino
      estado <- "Reprobado"
    FinSi
  FinSi
```

```
FinSi


Si avance < 60 Y estado = "Excelente" Entonces
   estado <- "Aprobado"
FinSi


observacion <- "Ninguna"

Si nota_final >= 7 Y doc = 0 Entonces
   observacion <- "Buen producto, pero mala formalidad"
FinSi


Escribir "======================="
Escribir "Promedio tecnico:", promedio
Escribir "Ajustes aplicados:", ajustes
Escribir "Nota final:", nota_final
Escribir "Estado:", estado
Escribir "Observacion:", observacion

FinSi

FinAlgoritmo
```

6) DIAGRAMA DE FLUJO


Algoritmo evaluacion_proyecto
Definir analisis, dise...
Definir avance, promed...
Definir errores Como E...
Definir doc, expo Como...
Definir ajustes Como Real
Definir estado, observ...
'Ingrese nota de anali...
analisis
'Ingrese nota de diseno:'
diseno
'Ingrese nota de codif...
codificacion
'Ingrese porcentaje de...
avance
'Ingrese numero de err...
errores
'Presento documentacio...
doc
'Realizo exposicion? (...
expo
analisis<0 0...
F
V
promedio ← (analisis+d...
nota_final ← promedio
ajustes ← 0
nota_final ← nota_fina...
ajustes ← ajustes-(err...
doc=1
nota_final ← nota_fina...
ajustes ← ajustes+0.5
expo=1
nota_final ← nota_fina...
ajustes ← ajustes+0.5
nota_final>10
nota_final ← 10
'Error: notas invalidas'



7) CODIGO EN C++

https://github.com/josephandrem/ALGORITMOS-Y-LOGICA-DE-PROGRAMACION-SW-1B/blob/main/main.cpp

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
   string nombreAlumno, Materia;
   double promedio, descuentoEr, error, nError, nota1, nota2, nota3, respuestaExp, RespuestaDoc, bonificacionExp, bonificacionDoc, notaNeta;
   bool notaValida1, notaValida2;

   error = 0.5;

   cout << "Ingrese el nombre del alumno" << endl;
   getline(cin, nombreAlumno);

   cout << "Ingrese la Asignatura" << endl;
```

```
    getline(cin, Materia);



do{
    cout << "Ingrese la nota - 1" << endl;
    cin >> nota1;

    cout << "Ingrese la nota - 2" << endl;
    cin >> nota2;

    cout << "Ingrese la nota - 3" << endl;
    cin >> nota3;

    notaValida1 = nota1 < 0 || nota2 < 0 || nota3 < 0;
    notaValida2 = nota1 > 10 || nota2 > 10  || nota3 > 10;

}while(notaValida1 || notaValida2);

    promedio = (nota1 + nota2 +nota3)/3;
    notaNeta = promedio;


    cout << "Ingresar el numero de errores cometidos" << endl;
    cin >> nError;

    descuentoEr = (nError*error);

    cout << "El alumno realizo la exposicion: si - 1 | no - 2" << endl;
    cin >> respuestaExp;

    bonificacionExp = 0;
    bonificacionDoc = 0;
    if(respuestaExp == 1){

        bonificacionExp = bonificacionExp + 0.5;

    }

    cout << "El alumno subio toda la documentacion: si - 1 | no - 2" << endl;
    cin >> RespuestaDoc;
```

```
if(respuestaExp == 1){

bonificacionDoc = bonificacionDoc + 0.5;

}

promedio = promedio - descuentoEr + bonificacionDoc + bonificacionExp;


bool excelente, aprobado, recuperacion, reprobado;

excelente = promedio <= 10 && promedio >=9;
aprobado = promedio <= 8.99 && promedio >=7;
recuperacion = promedio <= 6.99 && promedio >=5;
reprobado = promedio < 5;

string estado = "";

if(excelente){
   estado = "excelente";
}

   if(aprobado){
   estado = "aprobado";
}


   if(recuperacion){
   estado = "recuperacion";
}

   if(reprobado){
   estado = "reprobado";
}


cout << "El promedio del Alumno: " << nombreAlumno << " es - " << promedio << endl;
cout << "Estado: " << estado << endl;
   cout << "Ajustes aplicados: Errores: " << descuentoEr << " Documentacion: " << bonificacionDoc << "
Exposicion: " << bonificacionExp << endl;
if (bonificacionDoc == 0){
   cout << "Observacion: Buen producto, Mala formalidad";
```

```
    }

    return 0;
}
```

8) PRUEBAS DE SOLUCIÓN

Primer caso

Análisis	Diseño	Codificación	Avance	Errores	Documentación	Exposición	Prom	Ajustes	Nt final	Estado	Obs
9	9	10	90	0	1	1	9.33	+1	10	Excelente	N/A

Segundo caso

Análisis	Diseño	Codificación	Avance	Errores	Documentación	Exposición	Prom	Ajustes	Nt Final	Estado	Obs
8	8	8	85	3	1	1	8	-0.5	7.5	Probado	N/A

Tercer caso

Análisis	Diseño	Codificación	Avance	Errores	Documentación	Exposición	Prom	Ajustes	Nt final	Estado	Obs
6	6	6	70	1	0	0	6	-0.5	5.5	Recuperación	N/A

2.8 Habilidades blandas empleadas en la práctica

☒ Liderazgo
☐ Trabajo en equipo
☒ Comunicación asertiva
☐ La empatía
☒ Pensamiento crítico
☐ Flexibilidad

☐ La resolución de conflictos
☐ Adaptabilidad
☒ Responsabilidad

2.9 Conclusiones

- Integridad de datos: La validación de rangos (0 a 10) previene errores de cálculo y asegura que el promedio técnico sea confiable.
- Equilibrio evaluativo: El uso de ajustes (bonos y multas) permite que la nota final refleje tanto la calidad del código como la responsabilidad formal.
- Impacto del avance: La restricción del 60% demuestra que el cumplimiento del alcance es tan crítico para la excelencia como la capacidad técnica.

2.10 Recomendaciones

- Refinar validaciones: Implementar ciclos de repetición en el código para que el usuario no tenga que reiniciar el programa ante una nota errónea.
- Automatizar penalizaciones: Escalar la resta por errores de forma dinámica para desincentivar entregas con fallos críticos de funcionamiento.
- Digitalizar formalidades: Integrar un sistema de verificación de archivos para que la variable de documentación no dependa únicamente de la entrada manual.

2.11 Referencias bibliográficas

https://github.com/josephandrem/ALGORITMOS-Y-LOGICA-DE-PROGRAMACION-SW-1B